

Part 1. 비전 AI 지도 그리기

OpenCV vs YOLO vs MediaPipe — 오늘 하루를 위한 지도

같은 문제를 세 가지 다른 방식으로 푼다 — 그 차이를 아는 것이 오늘의 시작점입니다.

강사: 김생근

2026년 7월 · Part 1

왜 오늘 세 가지 방식을 한 번에 배우나요?

"비전 AI" 튜토리얼은 유튜브에도, 블로그에도 이미 넘칩니다. 대부분 "이 라이브러리로 이렇게 코드를 짜세요"에서 멈춥니다. 코드를 그대로 따라 치는 건 오늘 이어지는 Part 2~3에서 직접 하게 됩니다.

정작 부족한 건 코드가 아니라 지도입니다 — 내 앞의 비전 문제를 만났을 때 "이건 OpenCV로 충분한가, YOLO를 학습시켜야 하나, MediaPipe를 그대로 가져다 쓰면 되나"를 판단하는 눈입니다. 이 판단을 못 하면 번호판 인식에 얼굴 인식용 모델을 끌어오거나, 색상 하나 구분하는 문제에 딥러닝 모델을 학습시키는 식의 낭비가 생깁니다.

오늘 50분의 목표는 정확히 이 지도를 그리는 것입니다. 그리고 이 지도가 있어야 오늘 이어지는 Part 2(번호판 인식)·Part 3(MediaPipe 도전과제)·Part 4(맞해봐 워크스루)에서 "지금 어느 도구가, 왜 이 자리에 쓰였는지"가 보이기 시작합니다.

학습 목표

#	이 클립을 마치면
1	비전 문제를 검출 → 전처리 → 인식/분류 → 후처리 4단계 파이프라인으로 쪼개 볼 수 있다
2	OpenCV·YOLO·MediaPipe가 같은 문제를 푸는 방식의 차이를 설명할 수 있다
3	오늘 하루 실습(Part 2~5)이 어떤 순서·역할로 이어지는지 안다
4	"학습된 모델이 필요한 순간과 필요 없는 순간"을 가르는 기준을 가진다

개념 1. 파이프라인 사고 — 문제를 4단계로 쪼개기

🤔 왜 필요한가?

"이 문제는 무슨 라이브러리로 풀지?"를 먼저 물으면 코드부터 뒤지게 됩니다. "이 문제가 어떤 단계로 이루어지나?"를 먼저 물으면 도구는 자연스럽게 따라옵니다. 잠시 후 Part 2(번호판 인식) 실습이 바로 이 단계 분해로 설계되어 있습니다.

💡 비유로 이해하기

공장 조립 라인을 떠올려 보세요. 원자재(카메라 프레임)가 들어오면 ① 검사(어디에 대상이 있나) → ② 가공(다루기 좋은 형태로 다듬기) → ③ 완성(무엇인지 읽거나 분류하기) → ④ 포장(결과를 실제로 쓸 수 있게 정리하기) 순서를 거칩니다. 비전 파이프라인도 똑같습니다.

🔍 원리 설명

단계	하는 일	예시
① 검출 (Detection)	프레임 안에서 관심 대상의 위치를 찾는다	번호판이 어디 있나, 손이 어디 있나
② 전처리 (Preprocessing)	찾은 영역을 다루기 좋은 형태로 다듬는다	기울기 보정, 크기 정규화
③ 인식/분류 (Recognition)	실제 내용을 읽거나 분류한다	문자 인식(OCR), 손모양 판정
④ 후처리 (Postprocessing)	결과를 정제해 실제로 쓸 수 있게 만든다	오탐 제거, 포매팅

🤖 실무 관점

잠시 후 Part 2(번호판 인식)가 정확히 이 4단계로 설계되어 있습니다 — Stage1 YOLO 검출 → Stage2 OpenCV 전처리 → Stage3 OCR 인식 → Stage4 후처리. 하나의 파이프라인 안에 서로 다른 도구가 각자 잘하는 자리에 들어갑니다.

? 도구 하나만 잘하면 되는 거 아닌가요?

아니요, 정확히 반대입니다. 오늘 배울 세 도구(OpenCV·YOLO·MediaPipe)는 서로 경쟁하는 관계가 아니라, 파이프라인의 각기 다른 단계에 특화된 협력 관계입니다.

"어느 도구가 제일 좋은가"가 아니라 "이 단계엔 어느 도구가 맞는가"를 묻는 것 — 그것이 오늘 지도의 핵심입니다.

개념 2~4. 같은 문제, 세 가지 다른 방식

개념 2. OpenCV — 규칙을 사람이 적어주는 전통 영상처리

🤔 왜 필요한가 · 💡 비유

딥러닝 없이도 풀리는 문제에 딥러닝을 쓰면 무겁고 느려집니다. 색상 범위, 윤곽선처럼 규칙이 명확한 문제는 사람이 규칙을 다 적어주는 전통 기법이 더 가볍고 예측 가능합니다. OpenCV는 "정해진 매뉴얼대로 움직이는 숙련된 검수원"에 가깝습니다 — 색이 이 범위 안이면 피부, 윤곽선이 이렇게 꺾이면 손가락, 하는 식으로 규칙을 사람이 직접 짭니다.

🔍 원리 · 🤖 실무 관점

학습이 없으니 모델 파일이 없습니다(용량 0바이트). 규칙이 명확하고 배경이 안정적인 문제(문서 스캔, 색상 기반 분리)엔 강하지만, 규칙 밖의 변수(카메라 각도, 손 포즈, 조명)가 늘어나면 급격히 무너집니다 — 왜 무너지는지는 잠시 후 실험 결과 박스에서 실측으로 보여드립니다.

? OpenCV는 구식이라 이제 안 쓰나요?

아닙니다. 규칙이 뚜렷한 문제(문서 스캔의 기울기 보정, 색상 기반 1차 분리)에는 지금도 딥러닝보다 가볍고 빠릅니다. 잠시 후 Part 2 파이프라인의 Stage2(전처리)도 OpenCV가 맡습니다.

개념 3. YOLO — 데이터로 학습시키는 딥러닝 탐지

🤔 왜 필요한가 · 💡 비유

사람이 규칙으로 다 적을 수 없을 만큼 변수가 많은 문제(다양한 각도·조명·차종의 번호판)는 규칙 대신 수많은 예시 사진을 보여주고 스스로 패턴을 체득하게 만듭니다. YOLO는 "수많은 사례를 본 뒤 감을 잡은 신입"에 가깝습니다.

🔍 원리 · 🤖 실무 관점

이미지 한 장을 신경망에 한 번 통과시켜(You Only Look Once) 물체의 위치와 종류를 동시에 예측합니다. 목적에 따라 모델 크기를 고릅니다 — 실시간성이 중요하면 경량 모델(YOLO11n), 정확도가 우선이면 대형 모델(YOLO11x)을 씁니다. 잠시 후 Part 2(번호판 검출)의 Stage1이 YOLO를 씁니다.

? 그럼 무조건 큰 모델이 좋은 거 아닌가요?

아니요. 속도·용량·정확도는 트레이드오프입니다. 잠시 후 실험 결과 박스의 YOLO11n(5.5MB) vs YOLO11x(114.4MB) 비교가 그 격차를 보여줍니다 — 20배 넘게 차이 납니다.

개념 4. MediaPipe — 구글이 미리 학습해 둔 온디바이스 모델

🤔 왜 필요한가 · 💡 비유

손·얼굴처럼 이미 구글이 방대한 데이터로 학습해 둔 범용 대상은 처음부터 다시 학습시킬 필요가 없습니다. MediaPipe는 "이미 자격증을 딴 전문가를 그대로 채용하는 것"에 가깝습니다 — 내가 처음부터 가르칠 필요가 없습니다.

🔍 원리 · 🤖 실무 관점

손이면 21개 랜드마크처럼, 사전학습된 좌표를 그대로 돌려줍니다. 각 랜드마크에는 이름(예: 엄지=4번)이 붙어 있어 위치뿐 아니라 "어느 부위인가"까지 압니다. 손·얼굴·포즈처럼 범용 대상은 MediaPipe, 번호판처럼 도메인 특화 대상은 커스텀 YOLO — 대상이 얼마나 범용적인가가 선택 기준입니다.

? MediaPipe도 딥러닝인데 YOLO랑 뭐가 다른가요?

맞습니다, 둘 다 딥러닝입니다. 차이는 "누가 학습을 이미 해줬나"입니다 — MediaPipe는 구글이 손·얼굴 같은 범용 대상을 이미 학습해 배포하고, YOLO는 번호판처럼 내 도메인의 대상을 내가 학습(또는 파인튜닝)해야 합니다.

범용 대상엔 남이 학습해 둔 것을 쓰고, 내 도메인 대상엔 내가 학습시킨다 — 이것이 셋을 가르는 기준입니다.

실측으로 보는 3자 비교

세 방식이 "같은 문제"(가위바위보 손모양 판정)를 어떻게 다르게 푸는지, 이 워크샵 안에서 실제로 측정한 수치로 확인합니다.

실험 결과 — 가위바위보 판정 3자 비교

항목	OpenCV 전통기법	MediaPipe HandLandmarker	YOLO (11n / 11x)
학습된 모델 용량	[사실] 0 B	[사실] 7.8 MB	[사실] 5.5MB(11n) / 114.4MB(11x)
가위바위보 실사진 6장 정확도	[사실] 2/6 (33%)	[사실] 6/6	[사실] 4/6 (67%, 11x)
오분류 건수	[사실] 0건 (나머지는 전부 기권)	—	[사실] 2건 (가위→보 오판정, 확신도 0.87~0.88)
학습 필요 여부	없음	구글이 이미 학습	커스텀 학습·파인튜닝 필요

→ 실무 결론: 규칙이 명확한 문제엔 OpenCV, 범용 대상(손·얼굴)엔 MediaPipe, 도메인 특화 대상(번호판 등)엔 YOLO를 고릅니다. 단 YOLO는 학습 데이터가 실제 사용 조건을 덮는지 확인 없이는 '확신도 높은 오답'을 낼 수 있습니다(가위→보 오판정 사례 참고).

출처: [rps_3way_comparison.ipynb](#)(이 워크샵 part1_comparison/, 2026-07-30 실측 —

[opencv2_rps_classic-mediapipe1_hand_lite-yolo1_rps](#)의 기존 검증 코드를 그대로 불러와 동일한 6장에 재실행). 정확도 열 전부 이 노트북에서 직접 측정한 값입니다.

OpenCV가 놓친 4장은 전부 오분류가 아니라 "판정불가(기권)"였습니다. 특히 가위 포즈 실패는 파라미터를 아무리 조정해도(224개 조합 전수 탐색) 풀리지 않았습니다 — 볼록 껍질 결함에는 "어느 골이 엄지 쪽인지" 정보 자체가 없기 때문입니다. MediaPipe가 랜드마크로 엄지를 이름으로 지목할 수 있는 것과 정확히 대비되는 지점입니다.

오늘 할 것 미리보기

지도를 그렸으니 이제 오늘 하루가 어떻게 이어지는지 확인합니다. Part 2부터는 직접 손을 움직입니다.

시간	파트	형식	내용
10:30-12:30	Part 2. 번호판 인식	직접 빌드	Stage1 YOLO 검출 → Stage2 OpenCV 전처리 → Stage3 OCR → Stage4 후처리 → Streamlit 통합
13:30-14:30	Part 3. MediaPipe 도전과제	선택 실습	웹캠 실시간 손 감지를 MediaPipe로 붙여보고 YOLO 방식과 비교
14:40-16:00	Part 4. 맛해봐 워크스루	시연	데이터 수집 → YOLO 파인튜닝 → 게이트판정 → Flutter 배포까지 실제 문서·코드로 재현
16:00-16:40	Part 5. 정리	워크숍	내 도메인에 적용한다면 — 아이디어 스케치 + spec.md 작성 연습

1~2분 요약

1~2분 요약 — 이것만 기억하세요

1. 비전 문제는 검출 → 전처리 → 인식/분류 → 후처리 4단계로 쪼개 봅니다.
2. OpenCV = 규칙 기반(가볍다, 규칙 밖에서 무너진다). YOLO = 내 도메인 대상을 직접 학습(변수가 많은 문제). MediaPipe = 구글이 이미 학습해 둔 범용 대상을 그대로 사용.
3. 셋은 경쟁 관계가 아니라 협업 관계입니다 — 잠시 후 Part 2 실습이 이 협업을 직접 만들어 봅니다.

"어느 도구가 제일 좋은가"가 아니라 "이 문제엔 어느 도구가 맞는가" — 그 판단이 오늘 배운 지도입니다.

빠른 참조 카드

상황별 도구 선택

이런 상황이면	이 도구
규칙이 명확하고 가볍게 처리해야 한다 (문서 스캔, 색상 분리)	OpenCV
내 도메인에 특화된 대상을 탐지해야 한다 (번호판 등)	YOLO (커스텀 학습)
손·얼굴·포즈처럼 범용 대상을 빠르게 붙이고 싶다	MediaPipe

파이프라인 4단계

검출(어디 있나) → 전처리(다루기 좋게) → 인식/분류(무엇인가) → 후처리(쓸모있게)

모델 용량 한눈에

OpenCV	MediaPipe	YOLO11n	YOLO11x
0 B	7.8 MB	5.5 MB	114.4 MB

출처: [opencv2_rps_classic/README.md](#) · [mediapipe1_hand_lite/docs/](#) · [yolo1_rps/README.md](#) (2026-07-29 실측)